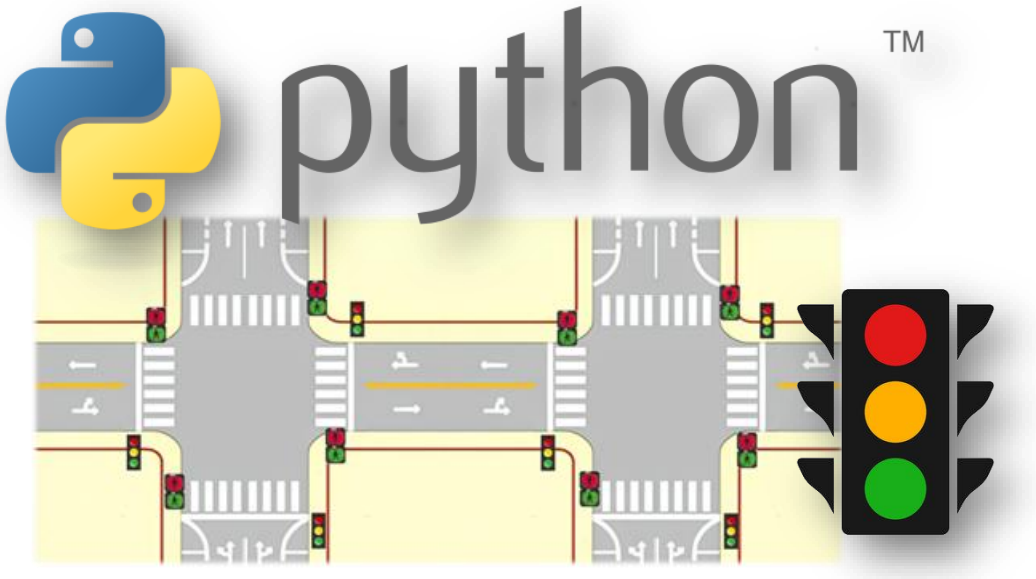


Trabalho 3 & 4

SISTEMAS OPERATIVOS - 2024/2025

André Lima	|	127168	|	12/06/2025
João Almeida	|	127140	|	12/06/2025

Cofinanciado pela União Europeia

Introdução

Este trabalho prático foi desenvolvido no âmbito da unidade curricular de Sistemas Operativos do CTeSP em RSI, com o objetivo de simular o funcionamento de um sistema de controlo de semáforos em dois cruzamentos. A consiste em implementar um programa baseado em comunicação por sockets, com múltiplas threads a interagir em tempo real. O sistema foi dividido em duas partes distintas, uma para o exercício 3 e outra para o exercício 4.

Na Parte 1, desenvolve-se um sistema cliente-servidor simples, em que os semáforos funcionam como clientes que comunicam diretamente com um servidor central. Este servidor gere os pedidos recebidos e controla o estado de cada semáforo de acordo com as regras definidas.

Na Parte 2, o código é reestruturado para ter em conta a introdução do Gestor, Servidores adicionais e um Distribuidor.

Índice

Introdução 1

Índice 2

- **Parte 1** 3
- **Parte 2** 7

Conclusão 15

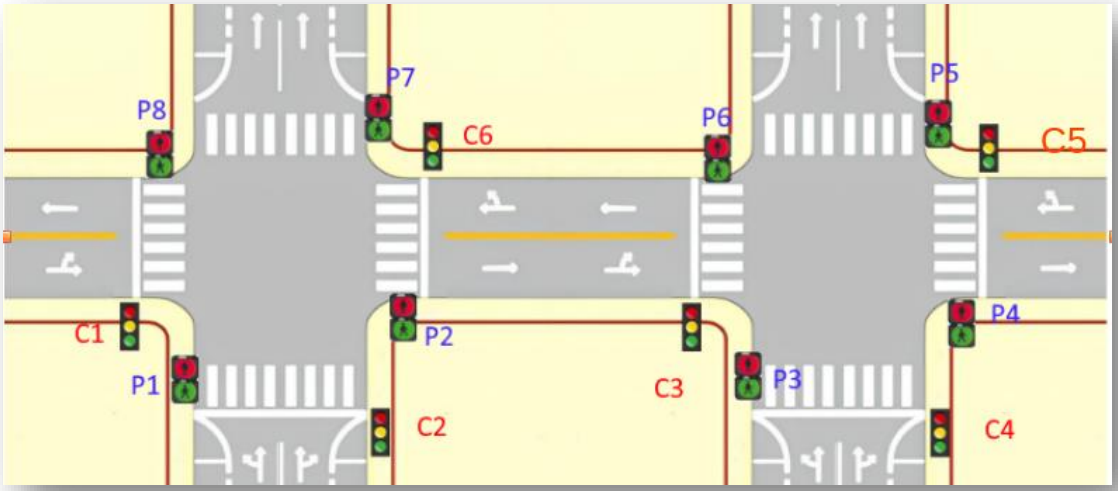
Parte 1

Na primeira parte do trabalho, foi implementado um sistema centralizado, onde todos os semáforos (clientes) se ligam a um único servidor. O objetivo principal desta fase foi compreender e aplicar os fundamentos da comunicação entre processos através de sockets, garantindo que cada semáforo envia a sua identificação e recebe instruções do servidor.

O servidor valida a ligação de todos os semáforos e apenas opera corretamente quando todas as ligações estão ativas, tal como indicado no enunciado. Além disso, é possível visualizar, de forma interativa, o estado atual dos semáforos, simulando o funcionamento de dois cruzamentos sincronizados.

Esta etapa serviu como base para a compreensão da lógica de controlo e gestão dos semáforos, permitindo consolidar os conceitos de redes, concorrência e gestão de estados.

Na imagem abaixo é possível ver o ID fornecido a cada semáforo, sendo os a vermelho os que representam os semáforos de carro e a azul os dos peões.



Código do Servidor

O servidor é o responsável por gerir a comunicação com os 14 semáforos (clientes) distribuídos em dois cruzamentos. O servidor aguarda a ligação de cada semáforo através de sockets TCP, controlando de forma alternada o fluxo de tráfego nas direções horizontal e vertical.

A cada ciclo de 10 segundos, o servidor determina o estado de cada semáforo com base na direção atual do tráfego e envia esta informação ao respetivo cliente. Adicionalmente, o programa apresenta periodicamente o estado de todos os semáforos conectados, incluindo se estão ativos e o seu estado atual (verde/vermelho).

Função determinar_estado

Esta função define o estado (VERDE ou VERMELHO) de um semáforo com base no seu identificador e na direção atual do tráfego (horizontal ou vertical). Semáforos horizontais como C1, C3, P1, P3, entre outros, ficam verdes quando o estado global é horizontal, e os verticais (C2, C4, P2, etc.) ficam verdes no estado vertical.

Função cliente_conectado

Executada por cada cliente em uma thread separada, esta função recebe o ID do semáforo conectado, valida se ele pertence à lista autorizada e se não está duplicado. Em seguida, adiciona a conexão ao dicionário global conexões_clientes.

Se houver erros ou ID inválido/duplicado, a conexão é terminada imediatamente.

Função mostrar_estado

Mostra o estado atual de todos os semáforos conectados. Divide a informação em dois cruzamentos distintos e indica:

- O tipo (Carro ou Peão);
- O estado (VERDE ou VERMELHO);
- A ligação (✓ ou ✗);
- Um ícone representando a cor atual (🟢 ou 🔴).

Utiliza a função linha_semáforo para formatar e exibir essas informações.

Função linha_semáforo

Função auxiliar dentro de mostrar_estado. Determina e organiza as informações de cada semáforo, como tipo, estado e conexão, retornando uma string formatada com esses dados.

Função servidor

É a função principal que inicializa o servidor e escuta as conexões dos semáforos. Controla a lógica do programa e alterna periodicamente o estado dos semáforos entre horizontal e vertical a cada 10 segundos, enviando atualizações a todos os clientes conectados.

Se todos os 14 semáforos estiverem conectados, o ciclo de envio e atualização começa. Se todos se desconectarem após já terem estado ligados, o servidor encerra automaticamente.

Função aceitar_clientes

Executada numa thread daemon, aceita conexões dos semáforos e cria uma nova thread para cada cliente usando cliente_conectado.

Código do Cliente

Cada cliente é executado individualmente e é responsável por estabelecer ligação ao servidor central, identificando-se através de um código único (como C1, P3, etc.). Uma vez conectado, o cliente permanece em escuta, aguardando instruções do servidor que determinam o seu estado atual (VERDE ou VERMELHO).

Função cliente_semaforo

A função principal do ficheiro, cliente_semaforo, é responsável por gerir a conexão entre o semáforo e o servidor. A função começa por criar um socket TCP e conecta-se ao servidor através do endereço IP local (127.0.0.1) e da porta 5000. Assim que a ligação é estabelecida, o cliente envia ao servidor a sua identificação, que é fornecida como argumento ao executar o ficheiro ou, caso não tenha sido indicado, é solicitada ao utilizador através de input manual.

Após o envio da identificação, o cliente entra num ciclo contínuo de receção de dados. A cada iteração, ele aguarda uma mensagem do servidor com o novo estado que o semáforo deve adotar. O estado (VERDE ou VERMELHO) é exibido no terminal junto com a identificação do semáforo.

Se, em qualquer momento, a ligação for interrompida (por exemplo, se o servidor for encerrado ou o semáforo for desconectado), o cliente deteta que não há mais dados a receber e encerra a conexão de forma segura. A função possui ainda tratamento de exceções para capturar e apresentar eventuais erros, garantindo que o socket é sempre encerrado corretamente, mesmo em caso de falha.

Por fim, fora da função principal, existe um pequeno bloco de verificação de argumentos no final do ficheiro. Este bloco permite que o utilizador forneça a identificação do semáforo diretamente na linha de comando ou, caso contrário, seja questionado interativamente. Em ambos os casos, essa identificação é passada para a função cliente_semaforo, permitindo que o cliente inicie corretamente a comunicação com o servidor.

Parte 2

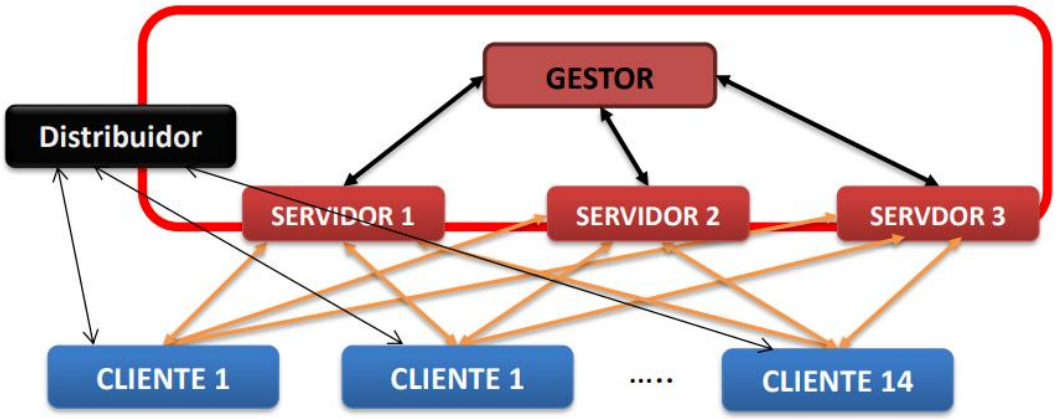
Na segunda parte do trabalho, a arquitetura inicial foi transformada num sistema distribuído, com maior modularidade e complexidade. Introduziram-se três novos componentes principais:

Gestor: elemento central que controla o estado global dos semáforos;

Servidores: intermediários entre o Gestor e os semáforos (clientes);

Distribuidor: responsável por encaminhar cada cliente para um dos servidores disponíveis.

Cada servidor tem um número limitado de clientes que pode aceitar em simultâneo, sendo os restantes colocados em fila de espera. O Gestor mantém o controlo central de todos os estados, e os servidores funcionam como pontos de distribuição e comunicação local.



Código do Gestor

O ficheiro gestor.py implementa a lógica de coordenação central do sistema de semáforos distribuídos. Este gestor atua como uma entidade responsável por receber conexões de múltiplos servidores (no máximo três), determinar os estados apropriados para cada semáforo e comunicar esses estados aos servidores correspondentes. A comunicação entre o gestor e os servidores é realizada através de sockets TCP, e o sistema é projetado de forma multithreaded para permitir o tratamento simultâneo das ligações. Além disso, o gestor atualiza ciclicamente o estado global dos semáforos entre "horizontal" e "vertical", garantindo uma alternância sincronizada e controlada do tráfego nos cruzamentos.

Função determinar_estado

A função determinar_estado serve para calcular o estado atual (VERDE ou VERMELHO) de um semáforo específico, com base no estado global do sistema (horizontal ou vertical). Dependendo da sentido do tráfego, alguns semáforos recebem sinal verde enquanto os outros permanecem em vermelho.

Função servidor_conectado

A função servidor_conectado é invocada cada vez que um servidor tenta estabelecer ligação ao gestor. Ao receber a conexão, o gestor solicita a identificação do servidor e valida se ela pertence ao conjunto de identificações válidas (SERVIDOR_1, SERVIDOR_2, ou SERVIDOR_3). Em caso de ID inválido ou repetido, a ligação é rejeitada. Caso contrário, o socket do servidor é armazenado numa estrutura de dicionário (conexões_servidores) que permite ao gestor enviar atualizações diretamente a cada servidor ativo. Esta função é executada em threads separadas, garantindo que múltiplos servidores podem ligar-se simultaneamente.

Função mostrar_estado

A função mostrar_estado tem como objetivo apresentar no terminal, de forma clara e organizada, o estado atual de todos os semáforos do sistema. Ela percorre cada semáforo dos dois cruzamentos e, para cada um, exibe o estado (VERDE ou VERMELHO), o tipo (carro ou peão), o servidor responsável e se está atualmente conectado ou não. Também são usados ícones visuais (🟢 para verde, 🔴 para vermelho e ✓/✗ para indicar conexão) para facilitar a leitura do estado geral do sistema. Esta função é chamada a cada ciclo de atualização, servindo como feedback visual ao operador do gestor.

Função gestor

A função principal do ficheiro, gestor, é responsável por orquestrar o funcionamento geral do sistema. Inicializa o socket principal de escuta na porta 6000 e começa a aceitar conexões dos três servidores esperados, criando para isso uma thread separada dedicada à aceitação de conexões. Após a ligação de todos os servidores, o gestor entra num ciclo contínuo onde, a cada 15 segundos, envia para cada servidor os estados atualizados dos semáforos que lhe estão atribuídos. A mensagem enviada é serializada no formato JSON, contendo um dicionário com os IDs dos semáforos e os respetivos estados.

Durante este processo, o gestor verifica se todos os servidores ainda estão conectados, removendo automaticamente os que apresentarem falhas de comunicação. Após cada ciclo de envio, é chamada a função mostrar_estado para atualizar visualmente o estado no terminal. Por fim, alterna-se o estado global do sistema de horizontal para vertical (e vice-versa), preparando o próximo ciclo de atualização.

Código do Distribuidor

Este ficheiro implementa o módulo distribuidor do sistema de semáforos. A função principal do distribuidor é receber conexões dos clientes (semáforos) e direcioná-los para o servidor responsável pela sua gestão. Com base no identificador do semáforo fornecido pelo cliente, o distribuidor consulta uma tabela de atribuição estática e envia ao cliente as informações necessárias para que este possa estabelecer a ligação correta com o servidor correspondente. O programa usa sockets TCP, threads e controlo de concorrência para garantir que múltiplos clientes possam ser processados simultaneamente.

Função encontrar_servidor_para_semáforo

Esta função tem como objetivo localizar o servidor responsável por um determinado semáforo, com base no seu identificador. O identificador do semáforo (por exemplo, "C3" ou "P5") é comparado com as listas de semáforos associadas a cada servidor disponível. Caso seja encontrada uma correspondência, é retornado o dicionário com as informações do servidor correspondente (ID, endereço IP e porta). Caso contrário, a função retorna None, indicando que o semáforo não é reconhecido no sistema.

Função cliente_conectado

A função cliente_conectado é executada numa thread separada para cada cliente que se liga ao distribuidor. Após o cliente enviar o seu identificador, a função procura o servidor correspondente através da função encontrar_servidor_para_semáforo. Se for encontrado um servidor válido, envia ao cliente uma string contendo o ID, o endereço IP e a porta do servidor, permitindo que o cliente se reconecte diretamente a ele. Em caso de erro (semáforo desconhecido ou exceções durante a comunicação), o cliente é informado e a ligação é encerrada.

Função mostrar_estado

A função mostrar_estado exibe no terminal o estado atual do distribuidor. Mostra o número total de clientes que já foram encaminhados e lista todos os servidores disponíveis, incluindo os semáforos atribuídos a cada um. Esta função serve de painel informativo para o operador, sendo chamada no arranque do sistema para apresentar a configuração inicial e facilitar o acompanhamento da sua operação.

Função distribuidor

A função distribuidor é o ponto de entrada principal deste módulo. Inicializa o socket TCP na porta 4000 e começa a escutar conexões de clientes, aceitando até 50 ligações simultâneas. Ao receber uma conexão, cria uma nova thread dedicada ao processamento desse cliente, chamando a função cliente_conectado. Esta abordagem garante que múltiplos clientes podem ser tratados em paralelo, sem bloquear o servidor principal. O loop principal do distribuidor também apresenta periodicamente mensagens com o número total de clientes processados, enquanto aguarda novas conexões. Por fim, o sistema permite o encerramento manual via KeyboardInterrupt, assegurando que o socket é corretamente fechado ao terminar.

Código do Servidor

Este ficheiro implementa um servidor de semáforos responsável por gerir um subconjunto de semáforos do sistema de trânsito inteligente. Existem três instâncias possíveis deste servidor (SERVIDOR_1, SERVIDOR_2, SERVIDOR_3), e cada uma é responsável por um grupo diferente de semáforos (carros e peões). A sua função é receber comandos do gestor central, armazenar localmente os estados dos semáforos, e reenviá-los em tempo real aos clientes (semáforos) conectados.

Ao iniciar, o servidor conecta-se ao gestor para registar-se, depois fica à escuta de clientes que chegam através da indicação do distribuidor. O servidor garante que apenas os semáforos que ele deve gerir se consigam ligar, utilizando a identificação dos mesmos. As comunicações são geridas com sockets TCP, sendo cada cliente tratado em threads separadas, permitindo escalabilidade e concorrência.

Função conectar_ao_gestor

Esta função estabelece a ligação inicial entre o servidor e o gestor. Utiliza a identificação do servidor (SERVIDOR_1, etc.) para se registar junto do gestor central, através de um socket TCP. Caso a conexão seja bem-sucedida, o ID é enviado para que o gestor reconheça qual servidor está a comunicar. Se a ligação falhar, o servidor imprime uma mensagem de erro e retorna False, impedindo a continuação do processo.

Função escutar_gestor

Responsável por escutar as mensagens enviadas pelo gestor, esta função corre num thread separado. Quando uma mensagem chega (em formato JSON), ela é convertida para um dicionário Python e os estados recebidos são armazenados em estados_semaforos_recebidos. Cada estado é, então, reenviado ao cliente correspondente (semáforo). Se um cliente estiver indisponível ou ocorrer um erro no envio, ele é removido da lista de conexões ativas. Esta função garante que o servidor atue como intermediário entre o gestor e os semáforos reais.

Função cliente_conectado

Trata da ligação de clientes semáforos ao servidor. Quando um cliente se conecta, o servidor recebe o seu ID e verifica se este pertence ao grupo que ele gere. Se não pertencer, envia uma mensagem de erro e fecha a conexão. Se o ID for válido e não estiver duplicado, o socket do cliente é guardado no dicionário conexões_clientes, permitindo enviar mensagens personalizadas a cada um. Esta função corre num thread separado por cliente, permitindo múltiplas conexões simultâneas.

Função mostrar_estado

Apresenta visualmente o estado atual de todos os semáforos sob responsabilidade deste servidor. Mostra o tipo de semáforo (carro ou peão), se está ou não conectado, e qual o estado atual (VERDE ou VERMELHO), com um ícone correspondente (🟢/🔴).

Função servidor

É o ponto de entrada principal do programa. Define qual o grupo de semáforos a ser gerido com base no servidor_id recebido (via argumento ou input). Inicializa a conexão com o gestor e cria os sockets necessários para ouvir as conexões dos semáforos. Também inicia threads para escutar o gestor e aceitar clientes. Entra num ciclo principal que periodicamente imprime o estado do sistema ou encerra o servidor caso todos os semáforos se desliguem.

Código do Cliente

O cliente ao iniciar, liga-se primeiro ao distribuidor para saber qual o servidor que lhe será atribuído.

Função cliente_semáforo

A função cliente_semáforo é responsável por toda a lógica do cliente. Começa por tentar conectar-se ao distribuidor (localizado no endereço 127.0.0.1 na porta 4000) e envia o seu identificador (por exemplo, "C1" ou "P4"). O distribuidor responde com os dados do servidor ao qual o cliente deve ligar-se, no formato ID:endereço:porta. A função então interpreta esta resposta e extrai as informações necessárias.

Caso a resposta do distribuidor contenha um erro (por exemplo, semáforo desconhecido), o cliente exibe a mensagem e encerra o processo. Caso contrário, tenta então estabelecer uma nova conexão com o servidor indicado, envia novamente a sua identificação e entra num ciclo de escuta.

Neste ciclo principal, o cliente fica à espera de mensagens do servidor. Cada mensagem representa o estado atual do semáforo (por exemplo, "VERDE" ou "VERMELHO"). O cliente imprime esta informação acompanhada de um contador para facilitar o seguimento das atualizações. Se a conexão com o servidor for interrompida, o cliente termina a execução informando o utilizador.

A função também inclui uma verificação de erros para lidar com falhas na comunicação tanto com o distribuidor quanto com o servidor. O uso do finally assegura que a ligação ao servidor é sempre encerrada corretamente, mesmo em caso de erro.

Na parte final do código, é verificado se o utilizador forneceu um argumento via linha de comandos. Caso contrário, apresenta uma lista de semáforos disponíveis (divididos por carros e peões) e solicita manualmente a inserção da identificação.

Conclusão

A realização deste trabalho prático permitiu aplicar e consolidar diversos conceitos fundamentais dos Sistemas Operativos, em particular no que diz respeito à comunicação entre processos, sincronização, gestão de estados e estruturação de sistemas distribuídos.

O projeto permitiu desenvolver experiência prática no que se deve à utilização de sockets e threads, onde a organização e coordenação entre os vários sistemas é essencial para o bom funcionamento do sistema.